

Deep Learning from Production to Deployment

Task 5: API Testing and Validation using Postman/Curl

Student Name: Sakshi Vijay Sataye

Roll No: 30

Class: Msc DSBD A Part 2

Subject: Deep Learning from Production to Deployment

Githuh Repository Link: https://github.com/saakshi5/Deep_Learning_Task5_API-Testing-and-Validation-using-Postman-Curl/tree/main

1. Introduction

API testing is the process of validating whether an Application Programming Interface correctly receives requests, processes data, and returns the expected response. In this task, a Flask REST API integrated with the pretrained MobileNetV2 model is tested using Postman and Curl.

2. Objective

- Develop a Flask prediction API.
- Test GET and POST endpoints.
- Validate JSON responses.
- Perform negative testing.
- Verify exception handling.

3. Model Details

Feature	Value
Model	MobileNetV2
Pretrained	ImageNet
Input Size	224 × 224
Classes	1000
Framework	TensorFlow

4. API Endpoints

Method	Endpoint	Purpose
GET	/	API Status
POST	/predict	Image Prediction

5. Testing Tools

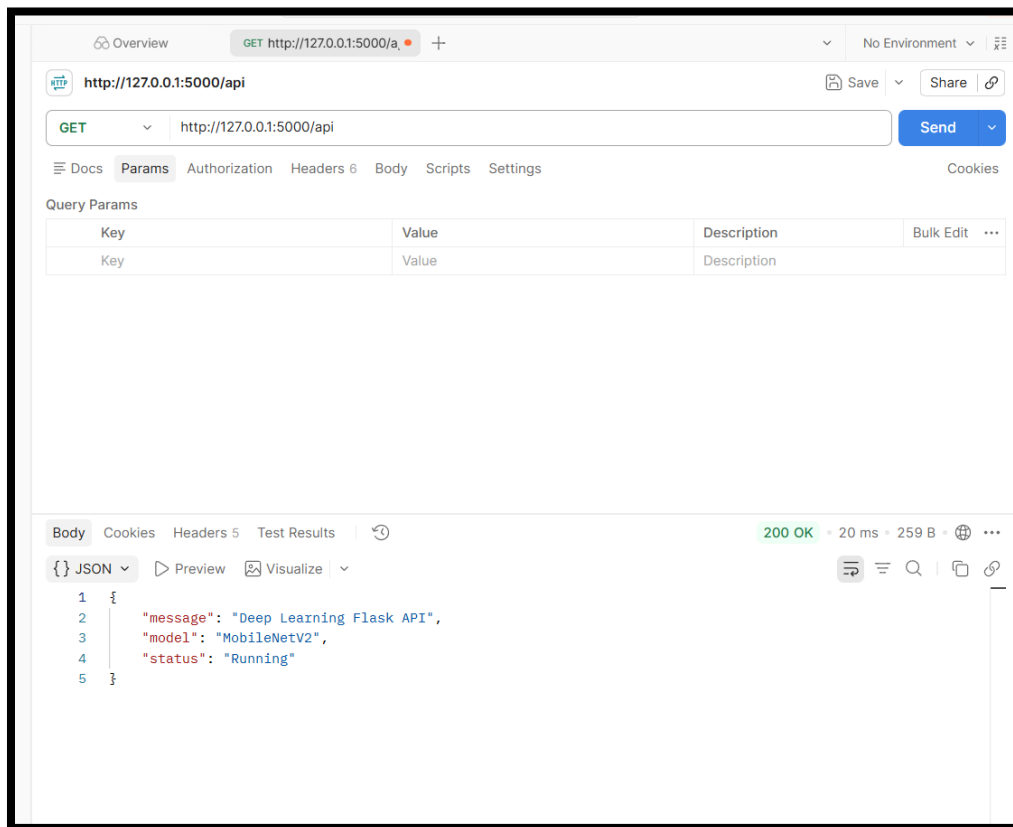
Tool	Purpose
Flask	REST API
Postman	API Testing
Curl	Command-line Testing
TensorFlow	Prediction Model

6. Test Cases

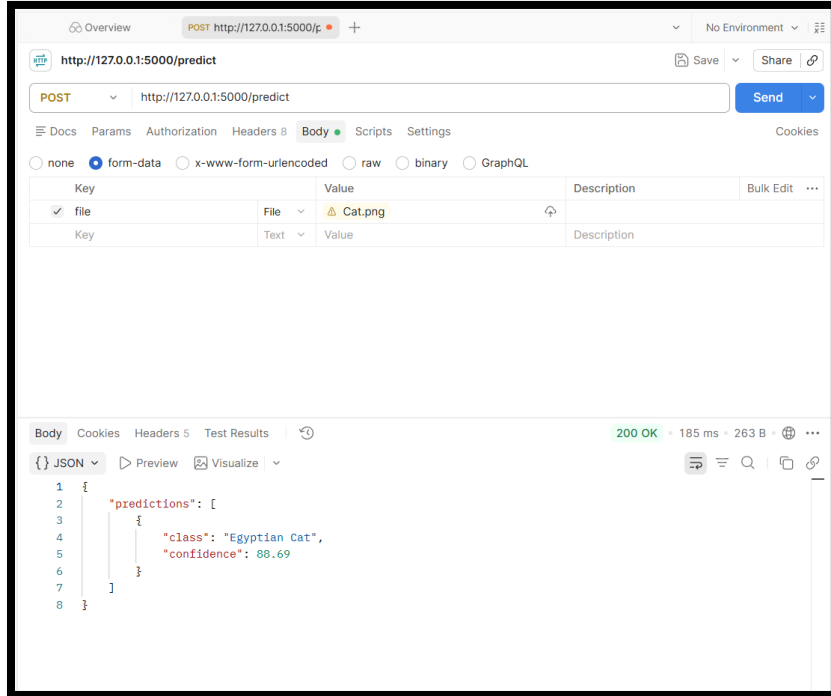
Test ID	Scenario	Expected Result
TC01	GET /	Status JSON
TC02	Valid Image	Prediction JSON
TC03	No File	Error 400
TC04	Empty Filename	Error Message
TC05	Curl Request	Successful Prediction

7. Postman Testing and Validation

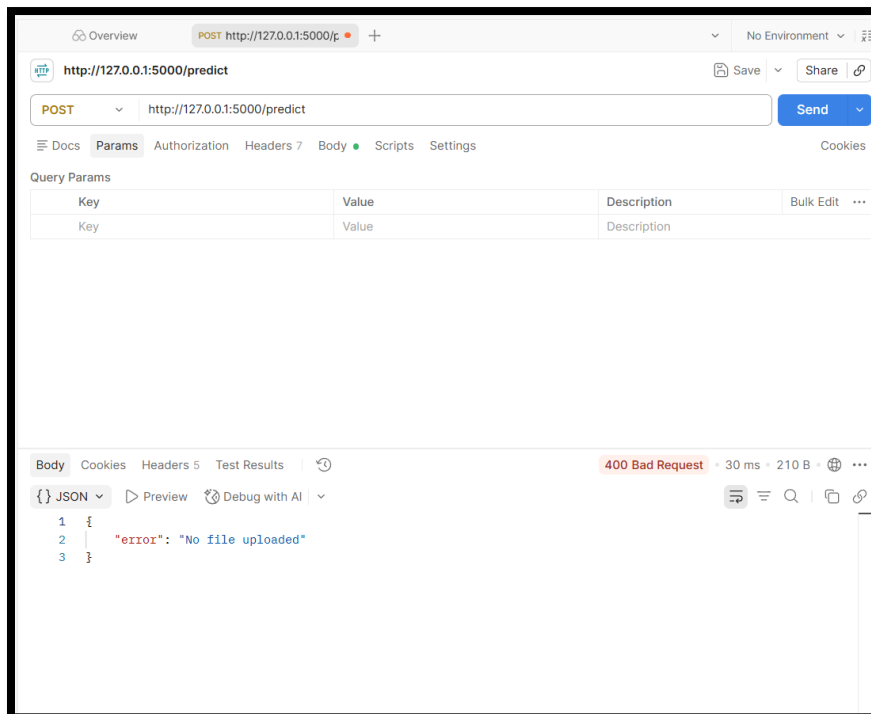
1. Browser displaying the JSON response from http://127.0.0.1:5000/.



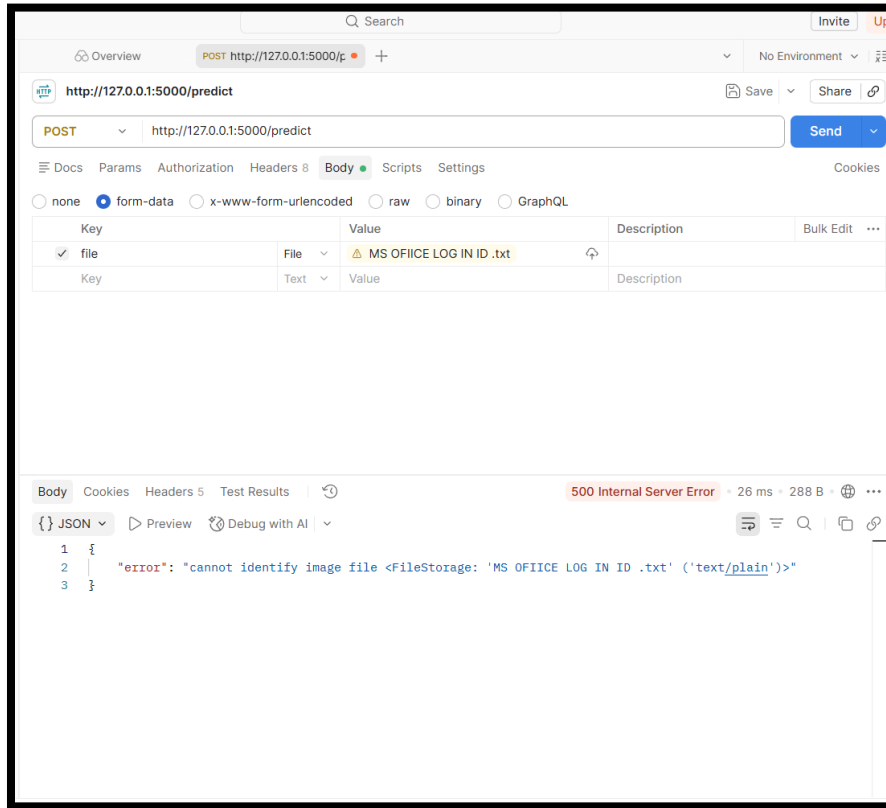
2. POST request with an uploaded image and the prediction JSON.



3. Postman API request resulting in a 400 Bad Request error because no file was uploaded in the request body.



4. **Postman API request resulting in a 500 Internal Server Error because a text file (.txt) was uploaded instead of an expected image file format.**



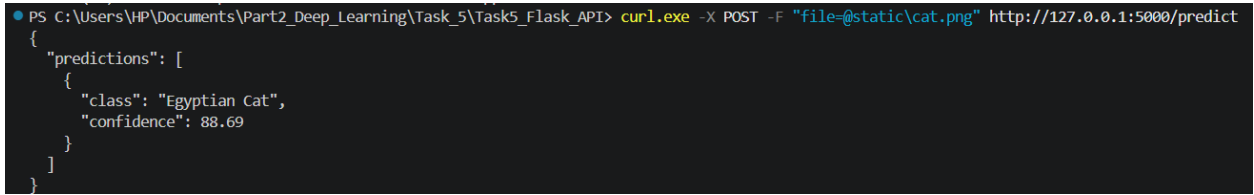
8. Curl Testing and Validation

1. Curl API Status

```
PS C:\Users\HP\Documents\Part2_Deep_Learning\Task_5\Task5_Flask_API> curl.exe http://127.0.0.1:5000/api
{
  "message": "Deep Learning Flask API",
  "model": "MobileNetV2",
  "status": "Running"
}
PS C:\Users\HP\Documents\Part2_Deep_Learning\Task_5\Task5_Flask_API>
```

Successful cURL request to a local Flask API, confirming that the MobileNetV2 deep learning model status is actively running.

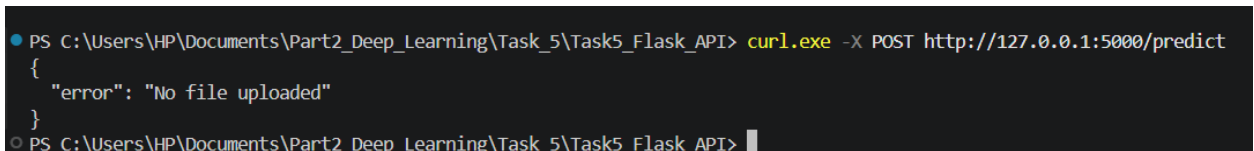
2. Curl prediction



```
PS C:\Users\HP\Documents\Part2_Deep_Learning\Task_5\Task5_Flask_API> curl.exe -X POST -F "file=@static\cat.png" http://127.0.0.1:5000/predict
{
  "predictions": [
    {
      "class": "Egyptian Cat",
      "confidence": 88.69
    }
  ]
}
```

The image shows a successful cURL POST request sending an image file (cat.png) to the /predict endpoint, resulting in a model prediction of "Egyptian Cat" with 88.69% confidence.

3. Curl Validation Testing



```
PS C:\Users\HP\Documents\Part2_Deep_Learning\Task_5\Task5_Flask_API> curl.exe -X POST http://127.0.0.1:5000/predict
{
  "error": "No file uploaded"
}
PS C:\Users\HP\Documents\Part2_Deep_Learning\Task_5\Task5_Flask_API>
```

The image shows a cURL POST request failing with a "No file uploaded" error response because no file payload was included in the command sent to the /predict endpoint.

9. Results

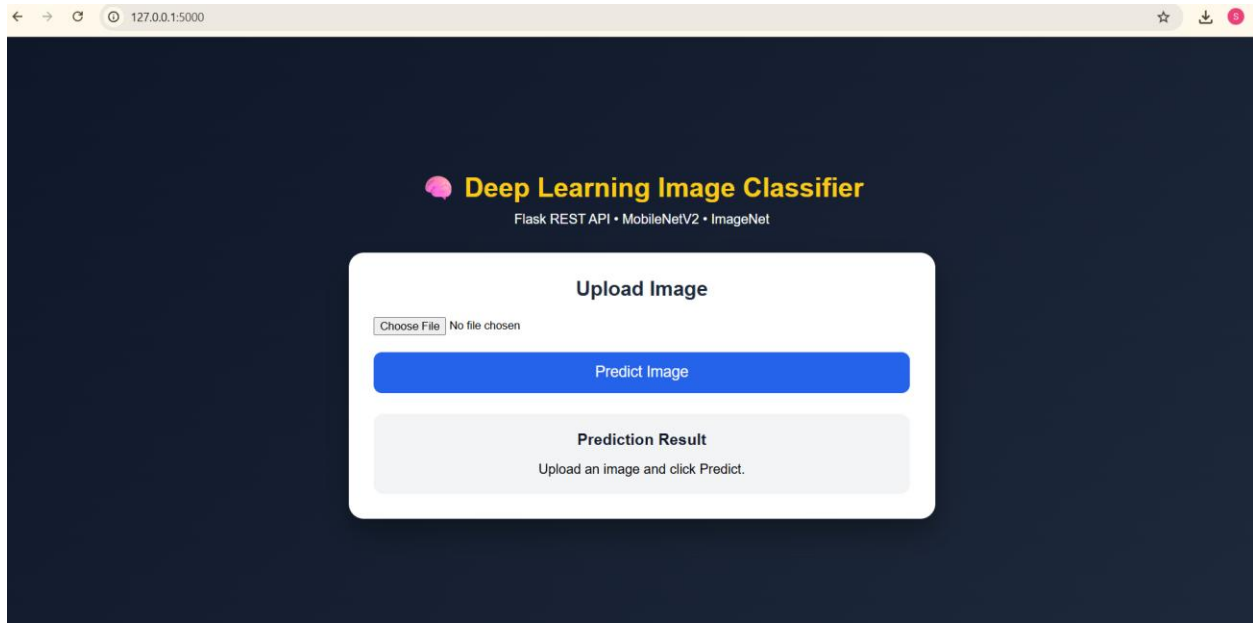
The Flask API successfully processed image uploads through the pretrained MobileNetV2 model, delivering identical, structured JSON responses across both Postman and cURL clients. Negative testing confirmed that the system is highly resilient, gracefully handling missing files and invalid formats with appropriate HTTP error codes without crashing.

10. Conclusion

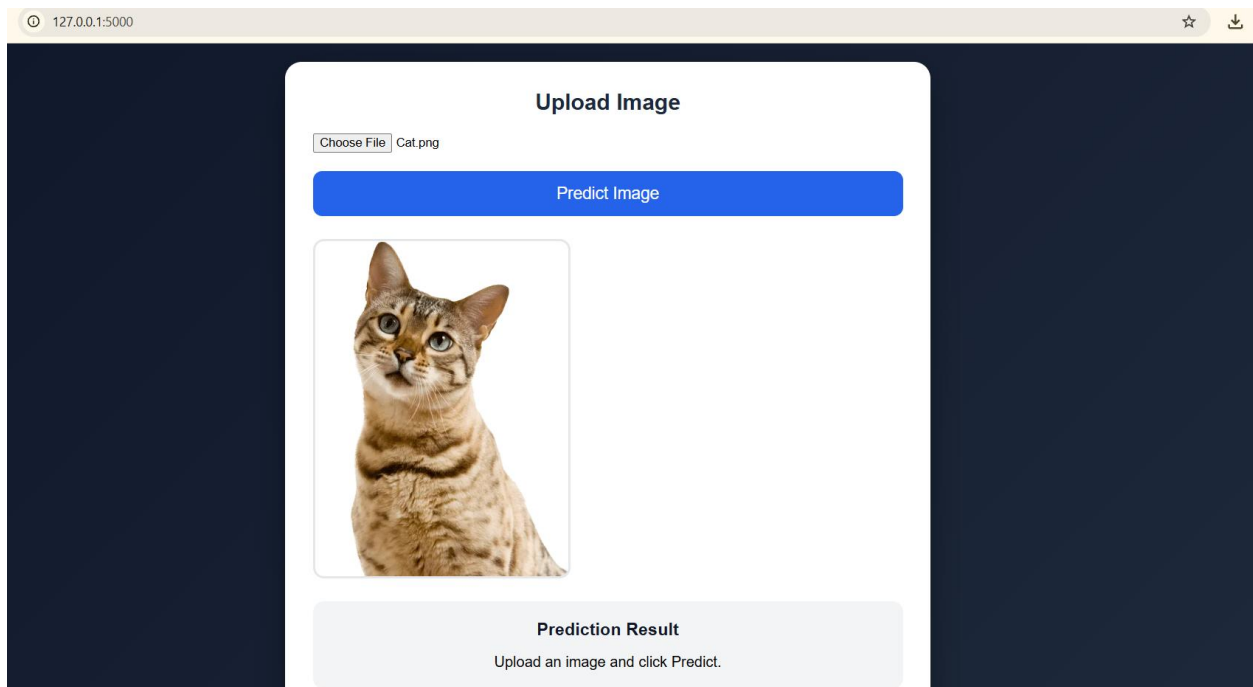
The project successfully demonstrates the implementation and complete validation of a production-ready, deep learning REST API utilizing the lightweight MobileNetV2 architecture trained on ImageNet weights. By integrating professional backend testing methods like cURL alongside an interactive frontend web user interface, the system achieves a thorough, full-stack verification of the machine learning inference pipeline. The core architecture relies on highly structured JSON data exchanges, enabling seamless client-server communication by cleanly packaging model predictions alongside precise statistical confidence percentages.

11.Task UI Screenshots

1. Home API (home_api.png)



2. API Prediction



Upload Image

Choose File Cat.png

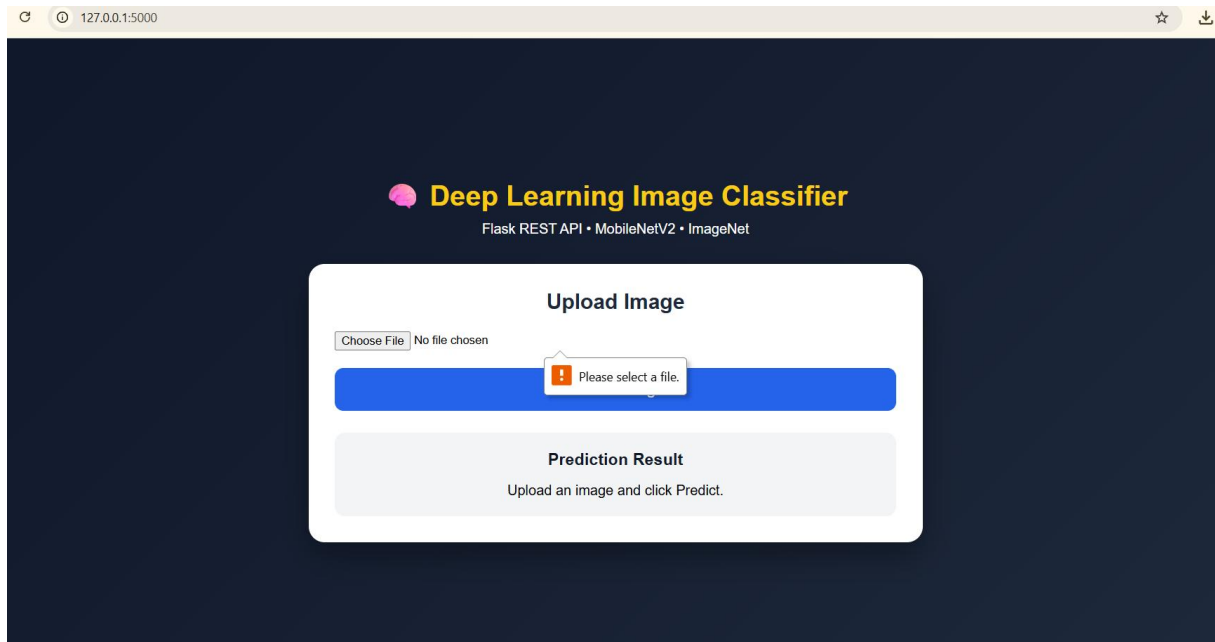
Predict Image



Predictions

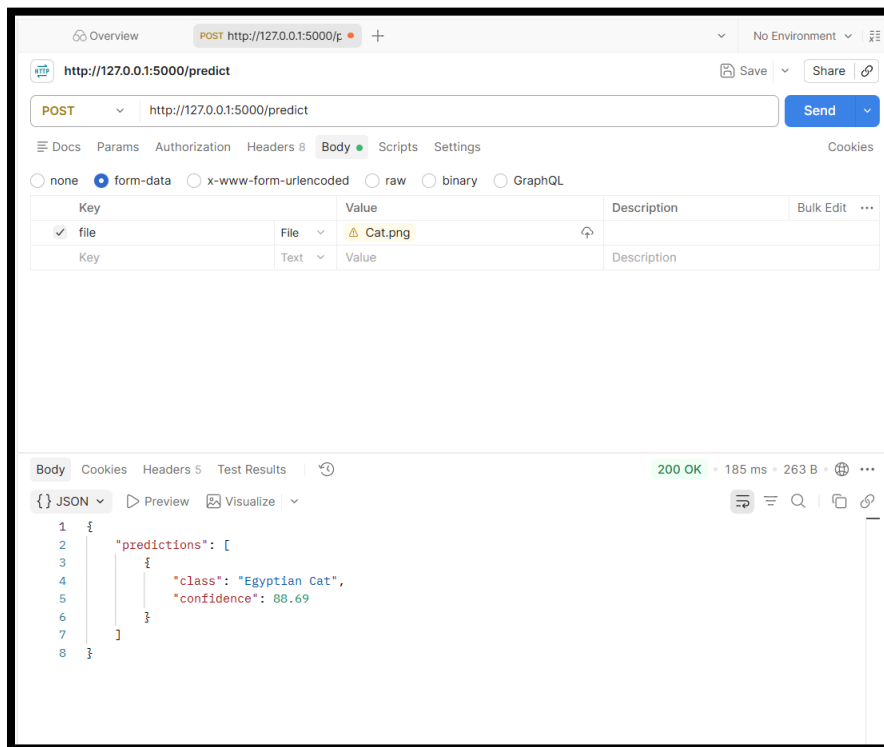
Egyptian Cat
88.69% Confidence

3. Frontend Validation for Empty Submission



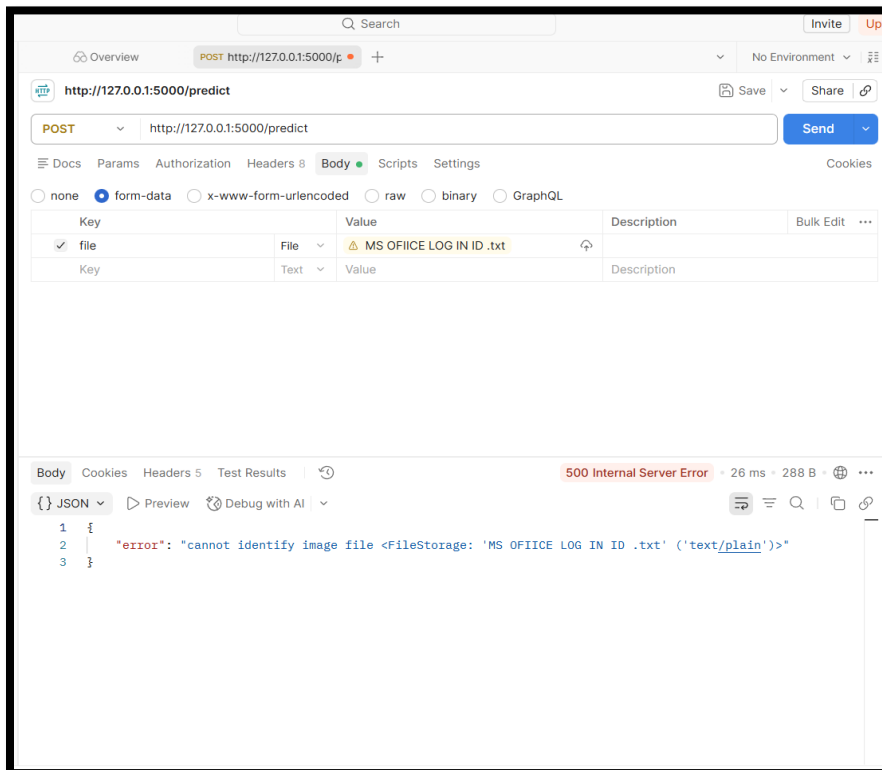
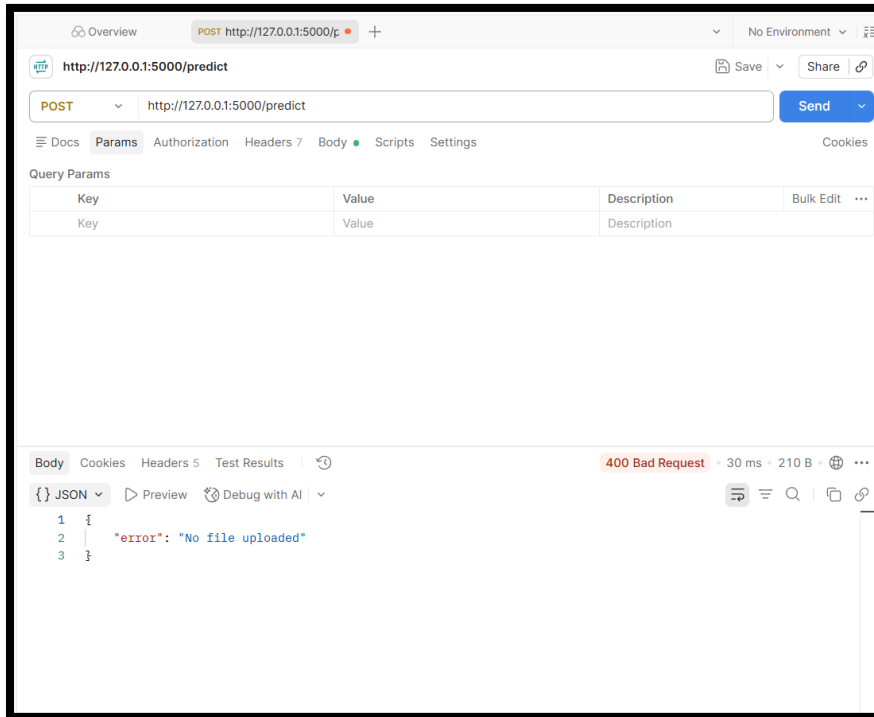
4. Successful Postman Test

POST request with an uploaded image and the prediction JSON.



5. Negative Testing

POST request without a file returning the error JSON.



6. Curl Testing

PowerShell showing the successful curl prediction response.

This satisfies the Task 5 deliverables: Flask API, Postman testing, Curl testing, response validation, negative testing, JSON logs, and documentation.

Curl Api Status

```
PS C:\Users\HP\Documents\Part2_Deep_Learning\Task_5\Task5_Flask_API> curl.exe http://127.0.0.1:5000/api
{
  "message": "Deep Learning Flask API",
  "model": "MobileNetV2",
  "status": "Running"
}
PS C:\Users\HP\Documents\Part2_Deep_Learning\Task_5\Task5_Flask_API>
```

Curl prediction

```
PS C:\Users\HP\Documents\Part2_Deep_Learning\Task_5\Task5_Flask_API> curl.exe -X POST -F "file=@static\cat.png" http://127.0.0.1:5000/predict
{
  "predictions": [
    {
      "class": "Egyptian Cat",
      "confidence": 88.69
    }
  ]
}
```

Curl Validation

```
PS C:\Users\HP\Documents\Part2_Deep_Learning\Task_5\Task5_Flask_API> curl.exe -X POST http://127.0.0.1:5000/predict
{
  "error": "No file uploaded"
}
PS C:\Users\HP\Documents\Part2_Deep_Learning\Task_5\Task5_Flask_API>
```